BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 10

Roll no: 28 PRN No: 12411933 Name of Student: **Pragati Rakhunde**

Subject Name : Deep Learning

1. Aim

To design and implement an audio application that combines speech-to-text (STT), text-to-speech (TTS), and deep-learning-based audio event classification, and to evaluate its behaviour using suitable metrics.

2. Objectives

- Capture spoken input and convert it into text using an STT component.
- Convert recognized text back into audible speech using TTS.
- Classify an audio file using a pre-trained YAMNet audio event classifier.
- Record execution time and prediction scores, and identify suitable dataset-level evaluation metrics.

3. Software and Hardware Requirements

Software: Python 3.10, Visual Studio Code, SpeechRecognition, TensorFlow, TensorFlow Hub, librosa, SoundFile, NumPy, and pyttsx3. Hardware: computer with microphone and speakers/headphones. Internet is required for Google Web Speech recognition and first-time model downloads.

4. Theory

Speech-to-Text (STT) converts a spoken audio signal into a text transcript. In this implementation, SpeechRecognition connects to Google's Web Speech recognition service; therefore, transcription requires internet access and is not performed by the local YAMNet model.

Text-to-Speech (TTS) synthesizes speech from text. The pyttsx3 library uses an installed system speech engine and can generally speak offline.

Dataset note: no large dataset is downloaded. For a lightweight demonstration, use one short 5–10 second WAV audio clip of a recognizable sound. For formal accuracy/precision/recall/F1 evaluation, use a small labelled set of several clips with known ground-truth labels. The pretrained YAMNet model weights are downloaded separately on first use.

YAMNet is a pre-trained deep neural network based on a MobileNet architecture that classifies audio events into a broad set of sound categories. It consumes mono audio sampled at 16 kHz and returns class scores over

audio frames. The application averages frame-level scores and reports the top five classes. The scores are model outputs, not a substitute for measured accuracy on a labelled test dataset.

5. System Workflow

1. For speech mode, capture audio from the microphone.
2. Send the recorded audio to the STT service and obtain a transcript.
3. Display the recognized text and measure the STT execution time.
4. Pass the transcript to TTS and play the synthesized speech.
5. For classification mode, load an audio file, convert it to mono 16 kHz audio, and run YAMNet.
6. Sort the class scores, display the top five predicted sound events, and record the runtime and scores in a CSV file.

6. Implementation

The complete implementation is provided in main.py. Install dependencies using requirements.txt. In VS Code PowerShell:

```
py -3.10 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Run microphone STT and TTS:

```
python main.py
```

Classify a local audio file:

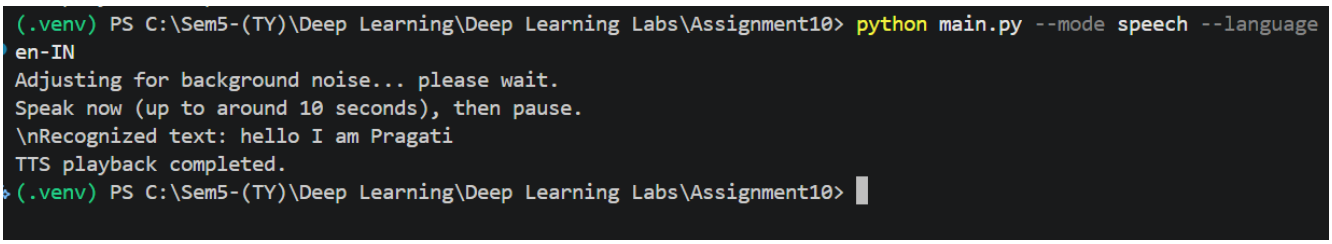
```
python main.py --mode classify --audio "dog_bark.wav"
```

7. Evaluation Metrics

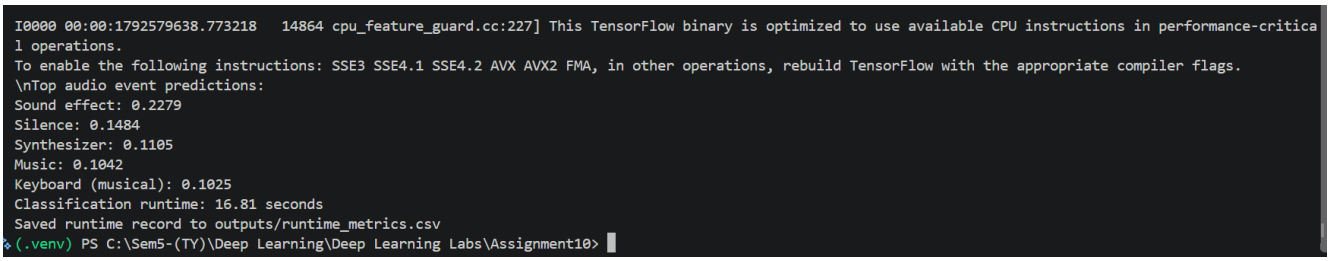
Metric	Definition / use
STT Word Error Rate (WER)	WER = (Substitutions + Deletions + Insertions) / Number of words in the reference transcript. Requires a manually verified reference transcript.
Classification accuracy	Correct predictions / Total labelled clips. Requires a labelled test set and a defined rule for converting model scores into predictions.
Precision, recall, and F1-score	Useful for class-wise evaluation when a labelled dataset is available, especially if classes are imbalanced.
Top-5 model scores	Shows the five highest-scoring YAMNet categories for an individual audio file; these are predictions, not dataset-level accuracy.
Runtime	Elapsed seconds for transcription or audio classification. First-run classification time may include downloading and loading model weights.

8. Screenshot Evidence

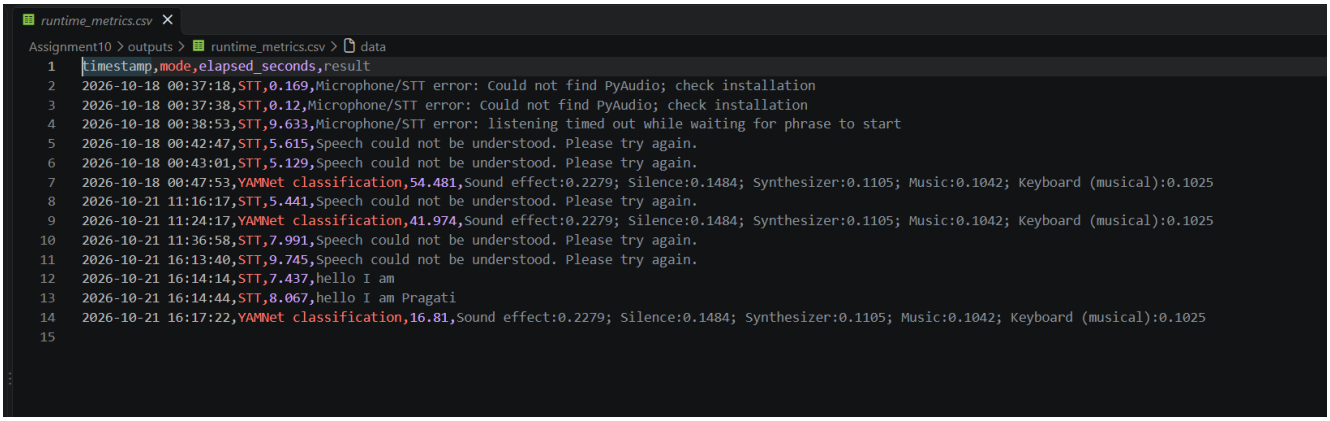
Screenshot 1: TTS playback execution / terminal confirmation.



Screenshot 2: Audio file classification with top-five predictions.



Screenshot 3: outputs/runtime_metrics.csv containing recorded runtime results.



9. Discussion

Discuss whether the STT transcript matches the spoken words, whether the synthesized speech is understandable, and whether the top audio-event predictions match the contents of the test clip. Background noise, microphone quality, accents, clip duration, and overlapping sound sources can affect performance. A single clip cannot establish overall classification accuracy. A reliable quantitative evaluation requires labelled audio samples and reference transcripts.

10. Limitations

STT uses an online recognition service and depends on internet access. YAMNet predicts general audio events rather than transcribing speech. TTS quality depends on installed system voices. The supplied starter records runtime and model scores but does not fabricate WER, accuracy, precision, recall, or F1 values.

11. Conclusion

The application integrates speech recognition, speech synthesis, and pre-trained deep-learning audio event classification in a Python project. It displays speech transcripts, speaks recognized text, and predicts likely audio-event categories for uploaded local audio files. Runtime and prediction scores are recorded automatically. Complete the final conclusion using actual test observations and, where available, WER and labelled-dataset classification metrics.

12. References

1) TensorFlow Hub YAMNet: https://tfhub.dev/google/yamnet/1

2) TensorFlow Models / YAMNet tutorial: https://www.tensorflow.org/hub/tutorials/yamnet

3) SpeechRecognition documentation: https://pypi.org/project/SpeechRecognition/

4) pyttsx3 documentation: https://pypi.org/project/pyttsx3/

5) TensorFlow documentation: https://www.tensorflow.org/